

Ever-Improving Test Suite by Leveraging Large Language Models

Ketai Qiu

Faculty of Informatics, Università della Svizzera Italiana (USI)

Lugano, Switzerland

ketai.qiu@usi.ch

Abstract

Augmenting test suites with test cases that reflect the actual usage of the software system is extremely important to sustain the quality of long lasting software systems. In this paper, we propose E-TEST, an approach that incrementally augments a test suite with test cases that exercise behaviors that emerge in production and that are not been tested yet. E-TEST leverages Large Language Models to identify *already-tested*, *not-yet-tested*, and *error-prone* unit execution scenarios, and augment the test suite accordingly. Our experimental evaluation shows that E-TEST outperforms the main state-of-the-art approaches to identify inadequately tested behaviors and optimize test suites.

CCS Concepts

• Software and its engineering → Software testing and debugging.

Keywords

Test Suite Augmentation, Field Testing, Large Language Models

ACM Reference Format:

Ketai Qiu. 2025. Ever-Improving Test Suite by Leveraging Large Language Models. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3696630.3728614>

1 Introduction

Test suites that sample the whole behavior of software systems in production can prevent sometime catastrophic failures, like the global service outages [19] that was due to a single software update from CrowdStrike and that resulted in a staggering \$5.4 billion loss. Unfortunately, it is impossible to generate a perfect test suite that exercises all scenarios and prevents all failures in production [8, 11].

Regression testing approaches maintain the test suites across incremental versions of the software systems by selecting and prioritizing the subsets of test cases to be re-executed on the new versions, and by relying on the former versions as test oracles [20, 22]. Field testing approaches leverage execution traces to test the actual behavior of software systems and enrich the test suites with scenarios from production [2, 18].

Ultimately, the set of execution traces observed in production is the best information about the actual use of a software system, since it includes all relevant scenarios, both the ones already tested

and the ones that the test suites have missed so far. However, simply adding all monitored scenarios to the test suite is clearly infeasible, due to the huge size of executions observed in production.

In this paper, we propose E-TEST, an effective approach to *continuously augment test suites without dramatically inflating the size of the suite, by identifying the not-yet-tested execution scenarios, that is, the subset of scenarios that are observed in production and that require additional testing*. E-TEST monitors the execution of the software system at the unit level, classifies the scenarios monitored in production as *already-tested*, *not-yet-tested* or *error-prone*, and generates test cases from the not-yet-tested scenarios to improve the quality of the test suite. E-TEST leverages a fine-tuned LLM to classify unit execution scenarios through five queries and generate test cases via limited rounds of chat interactions.

2 Related Work

Test suite augmentation is an emerging research area in last few years, as a way to enhance test suites with new test cases that can identify potential failures [1, 4, 5, 7]. Cruciani et al. [6] defined *FAST++*, an approach that uses K-means clustering of test case vectors to reduce the size of the test suite. *FAST++* works only for granular class-wise test suites and does not consider intrinsic code structures and relations. Shimmi and Rahimi [23] studied common patterns of co-evolution between source code and test suites, to enable the remediation of the test suite. Their approach strongly relies on manually summarized patterns. E-TEST improves the test suite with new scenarios that emerge in production, by excerpting the not-yet-tested scenarios from the execution traces using LLMs. E-TEST overcomes the limitations of state-of-the-art testing approaches by integrating a comprehensive analysis of source code, test cases, and execution traces thanks to strong code understanding capabilities learned from extensive training data.

Field testing leverages information extracted from the field using both static and dynamic analysis, to test the software systems with not-yet-tested scenarios [2]. Gazzola et al. [10] introduced *field-ready testing*, which executes emerging scenarios directly in production environment via instantiating parametric test case templates with data arising in production. E-TEST selects the scenarios that are observed in production and requires additional testing without actual execution, while *field-ready testing* focuses on executing scenarios in production, that interferes with the running software.

3 Approach

E-TEST monitors the input data *In* of the focal methods *M*, by executing the instrumented focal methods in production, and classifies *In* as *already-tested*, *not-yet-tested* or *error-prone*, with respect to a test suite *T*, in three steps: PREPROCESSOR, ANALYZER and POSTPROCESSOR as shown in Figure 1. Specifically, we built a dataset from Defects4J and four Java projects on GitHub [12] by instrumenting



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion '25, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06
<https://doi.org/10.1145/3696630.3728614>

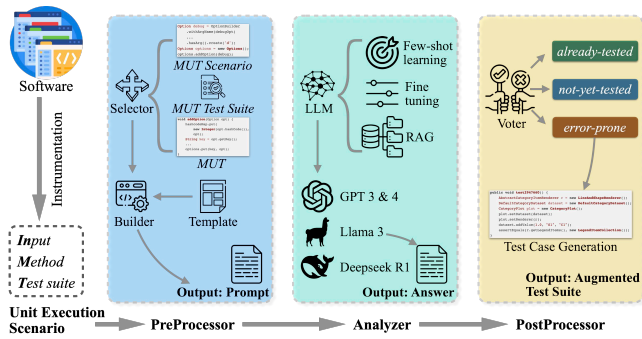


Figure 1: Overview of E-TEST.

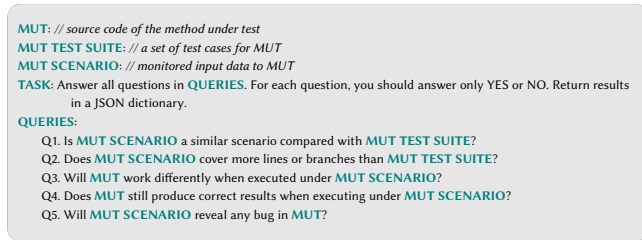


Figure 2: Prompt template.

bug-revealing test cases. We provide the dataset and evaluation results as a replication package ¹.

The PREPROCESSOR generates the prompt for the ANALYZER by instantiating the template with five sections: *MUT*, *MUT Test Suite*, *MUT Scenario*, *Task* and *Queries* as shown in Figure 2. The five questions are formulated with the most suited terms about software bugs from Stack Overflow [17].

The ANALYZER queries an LLM to answer the prompt with advanced configurations, including few-shot learning [24], Retrieval Augmented Generation (RAG) [14], and fine-tuning [25]. We leveraged LlamaIndex [15] to build indices for all source code and tests in the examined Java project and fine-tuned GPT 3.5 Turbo using OpenAI APIs and 5% of dataset. We experimented the ANALYZER with various LLMs using Ollama and OpenAI APIs, including Llama3 (8B and 70B), Deepseek-r1 70B, GPT (3.5 Turbo, 4 Turbo, 4o and fine-tuned 3.5 Turbo).

The POSTPROCESSOR classifies the input *In* as *already-tested*, *not-yet-tested* or *error-prone*, by combining the answers of the ANALYZER: 10010 = *already-tested*, 01010 = *not-yet-tested*, 01101 = *error-prone*, where 1 and 0 corresponds to a Yes and No answer to the query, respectively. The classification depends on the best partial matching of the five answers. For example, the POSTPROCESSOR classifies an input *In* with 10011 answers as *already-tested*, by counting the matchings. In case of a tie, *error-prone* dominates *not-yet-tested* that dominates *already-tested*.

4 Results

We evaluated the effectiveness of E-TEST on a dataset of 1,975 unit execution scenarios (537 *already-tested*, 719 *not-yet-tested*, and 719

¹<https://github.com/ketaiq/etest-replication-package-fse25src>

Table 1: Precision (P), Recall (R) and F1-score (F1) of 3-class classification of scenarios using the experimented LLMs (%)

LLM	<i>already-tested</i>			<i>not-yet-tested</i>			<i>error-prone</i>			Avg. F1
	P	R	F1	P	R	F1	P	R	F1	
Llama3 8B	26	19	22	36	51	42	36	27	31	32
Llama3 70B	18	29	22	34	28	31	28	19	23	25
Deepseek-r1 70B	18	29	22	29	24	26	52	45	48	32
GPT-3.5 Turbo	26	10	15	35	64	46	36	23	28	30
GPT-4 Turbo	17	20	18	34	56	42	31	7	11	24
GPT-4o	19	33	24	32	38	16	37	10	35	25
GPT-4 Turbo (3-shot)	19	24	21	35	52	42	32	9	14	26
GPT-4 Turbo (6-shot)	15	19	17	35	55	43	32	8	13	24
GPT-4 Turbo (9-shot)	17	24	20	34	54	42	30	4	7	23
Llama3 8B with RAG	23	28	26	43	51	47	53	37	44	39
Llama3.3 70B with RAG	22	51	30	34	22	27	62	32	42	33
GPT-3.5 Turbo with RAG	19	8	11	42	71	53	60	40	48	37
GPT-4 Turbo with RAG	15	34	21	31	33	32	55	11	19	24
Cruciani et al. [6]	32	32	32	27	27	27	44	44	44	34
Gazzola et al. [10]	50	100	67	0	0	0	18	33	23	30
E-TEST	66	94	78	47	24	31	48	57	53	54
E-TEST with RAG	47	41	44	51	61	56	52	46	49	50

error-prone scenarios) extracted from 17 Defects4J [13] projects and four popular open-source Java projects (Spring Boot [3], Apache ShardingSphere [21], Apache Dolphinscheduler [9] and Micrometer [16]). We split the Defects4J dataset into fine-tuning and validation sets with a 5:95 ratio. We fine-tuned GPT-3.5 Turbo with batch size 1, learning rate multiplier 2.0 and 3 epochs using the balanced fine-tuning dataset. We validated E-TEST on both the validation set from Defects4J and 150 scenarios from four GitHub projects. We measured the effectiveness of E-TEST as the amount of inputs that E-TEST correctly classifies as *already-tested*, *not-yet-tested* or *error-prone*, with respect to the available test suites. We compared E-TEST (instantiated on fine-tuned GPT-3.5 Turbo) with different LLMs both with and without RAG, and with the two closest approaches *FAST++* [6] and *field-ready testing* [10]. Table 1 reports the precision, recall and F1-score of each class for E-TEST with and without RAG (last two rows), the LLMs (top rows in the table), and *FAST++* and *field-ready testing* (rows Cruciani et al. [6] and Gazzola et al. [10]). E-TEST performs significantly better than the other LLMs, and largely outperforms both *FAST++* (+20%) and *field-ready testing* (+24%). Furthermore, a case study on test case generation reveals that the identified *error-prone* scenarios can detect 83.5% of failures. The experimental results demonstrate E-TEST’s significant potential to enhance both test suite quality and software reliability.

5 Contributions

This paper contributes to software testing by proposing (i) a novel viewpoint for long-lasting test suite augmentation using untested execution scenarios, (ii) E-TEST, an effective approach to classify unit execution scenarios, and generate failure-revealing test cases using LLMs, (iii) a dataset of 1,975 unit execution scenarios that we created from popular Java projects, (iv) a solid experimental evaluation of E-TEST with widely-used LLMs under different settings and comparison with two state-of-the-art approaches.

Acknowledgments

This work is supported by the Swiss SNF project A-Test Autonomic Software Testing (SNF 200021_215487).

References

- [1] Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (Porto de Galinhas, Brazil) (FSE 2024). Association for Computing Machinery, New York, NY, USA, 185–196. <https://doi.org/10.1145/3663529.3663839>
- [2] Antonia Bertolino, Pietro Braione, Guglielmo De Angelis, Luca Gazzola, Fitsum Kifetew, Leonardo Mariani, Matteo Orrù, Mauro Pezzè, Roberto Pietrantuono, Stefano Russo, and Paolo Tonella. 2021. A Survey of Field-Based Testing Techniques. *ACM Comput. Surv.* 54, 5, Article 92 (may 2021), 39 pages. <https://doi.org/10.1145/3447240>
- [3] Spring Boot. 2025. <https://github.com/spring-projects/spring-boot> Accessed: February 7, 2025.
- [4] Carolin Brandt, Ali Khatami, Mairieli Wessel, and Andy Zaidman. 2024. Shaken, Not Stirred: How Developers Like Their Amplified Tests. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1264–1280. <https://doi.org/10.1109/TSE.2024.3381015>
- [5] Carolin Brandt and Andy Zaidman. 2022. Developer-centric test amplification: The interplay between automatic generation human exploration. *Empirical Softw. Engg.* 27, 4 (July 2022), 35 pages. <https://doi.org/10.1007/s10664-021-10094-2>
- [6] Emilio Cruciani, Breno Miranda, Roberto Verdecchia, and Antonia Bertolino. 2019. Scalable Approaches for Test Suite Reduction. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 419–429. <https://doi.org/10.1109/ICSE.2019.00055>
- [7] Benjamin Danglot, Oscar Vera-Perez, Zhongxing Yu, Andy Zaidman, Martin Monperrus, and Benoit Baudry. 2019. A snowballing literature study on test amplification. *Journal of Systems and Software* 157 (2019), 110398. <https://doi.org/10.1016/j.jss.2019.110398>
- [8] Giovanni Denaro, Noura El Moussa, Rahim Heydarov, Francesco Lomio, Mauro Pezzè, and Ketai Qiu. 2024. Predicting Failures of Autoscaling Distributed Applications. *Proc. ACM Softw. Eng.* 1, FSE, Article 87 (July 2024), 22 pages. <https://doi.org/10.1145/3660794>
- [9] Apache Dolphinscheduler. 2025. <https://github.com/apache/dolphinscheduler> Accessed: February 7, 2025.
- [10] Luca Gazzola, Leonardo Mariani, Matteo Orrù, Mauro Pezzè, and Martin Tappler. 2022. Testing Software in Production Environments with Data from the Field. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 58–69. <https://doi.org/10.1109/ICST53961.2022.00017>
- [11] Luca Gazzola, Leonardo Mariani, Fabrizio Pastore, and Mauro Pezzè. 2017. An Exploratory Study of Field Failures. In *2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE)*. 67–77. <https://doi.org/10.1109/ISSRE.2017.10>
- [12] GitHub. 2025. <https://github.com/> Accessed: February 7, 2025.
- [13] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) (ISSTA 2014). Association for Computing Machinery, New York, NY, USA, 437–440. <https://doi.org/10.1145/2610384.2628055>
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [15] LlamaIndex. 2025. <https://www.llamaindex.ai/> Accessed: February 7, 2025.
- [16] Micrometer. 2025. <https://github.com/micrometer-metrics/micrometer> Accessed: February 7, 2025.
- [17] Stack Overflow. 2025. <https://stackoverflow.com/questions> Accessed: February 7, 2025.
- [18] Ketai Qiu. 2024. Autonomic Testing: Testing with Scenarios from Production. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings* (Lisbon, Portugal) (ICSE-Companion '24). Association for Computing Machinery, New York, NY, USA, 156–158. <https://doi.org/10.1145/3639478.3639802>
- [19] Oliver Rodzianko. 2024. CrowdStrike Is Significantly Undervalued Following Operational Crisis. <https://www.forbes.com/sites/gurufocus/2024/08/09/crowdstrike-is-significantly-undervalued-following-operational-crisis/> Accessed: January 20, 2025.
- [20] Gregg Rothermel, Sebastian Elbaum, Alexey G. Malishevsky, Praveen Kallakuri, and Xuemei Qiu. 2004. On test suite composition and cost-effective regression testing. *ACM Trans. Softw. Eng. Methodol.* 13, 3 (July 2004), 277–331. <https://doi.org/10.1145/1027092.1027093>
- [21] Apache ShardingSphere. 2025. <https://github.com/apache/shardingsphere> Accessed: February 7, 2025.
- [22] August Shi, Tiffany Yung, Alex Gyori, and Darko Marinov. 2015. Comparing and combining test-suite reduction and regression test selection. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering* (Bergamo, Italy) (ESEC/FSE 2015). Association for Computing Machinery, New York, NY, USA, 237–247. <https://doi.org/10.1145/2786805.2786878>
- [23] Samiha Shimmi and Mona Rahimi. 2022. Patterns of Code-to-Test Co-evolution for Automated Test Suite Maintenance. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 116–127. <https://doi.org/10.1109/ICST53961.2022.00023>
- [24] Yaqing Wang, Quanming Yao, James T. Kwok, and Lionel M. Ni. 2020. Generalizing from a Few Examples: A Survey on Few-shot Learning. *ACM Comput. Surv.* 53, 3, Article 63 (June 2020), 34 pages. <https://doi.org/10.1145/3386252>
- [25] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2025. Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). <https://doi.org/10.1145/3714461> Just Accepted.